Chapter 00 -- EKA Formalized: The Executable Knowledge Architecture Definition

- 0.0 Why This Chapter Exists
- 0.1 The Problem That EKA Solves
- 0.2 EKA -- A Working Definition
  - 0.2.1 Short Definition
  - 0.2.2 Formal Definition
- 0.3 The Five Layers of EKA (Implementation Roadmap)
- 0.4 Why "Executable" Is Not the Same as "Reasoning"
- 0.5 EKA in Relation to Existing Standards and Architectures
- 0.6 EKA Maturity Levels
- 0.7 EKA and the `Pizza.owl` Tutorial
- 0.8 How to Use the EKA Definition in Your Work
- 0.9 Key Concepts
- 0.10 Chapter (00) Summary
- 0.11 Reference

0.0 Why This Chapter Exists

If you have already read the preface or browsed the book's outline, you have encountered the acronym **EKA** -- Executable Knowledge Architecture.

You have seen it in:

- The EKA Implementation Roadmap diagram (repeated across chapters)
- "EKA Connection" sections at the end of most chapters
- References to `https://xiaoqi.com`

But until now, we have not paused to define EKA **formally**.

This chapter changes that.

By the end of this chapter, you will not only understand what EKA is - you will be able to:

- Distinguish EKA from ordinary ontology projects, knowledge graphs, and rule engines.
- Identify the **five core layers** of an executable knowledge architecture
- Recognize why **executability** transforms semantic modeling into operational (executable) intelligence
- Position the `Pizza.owl` tutorial within the larger EKA maturity roadmap

If you are a practitioner, architect, or researcher, this chapter gives you the **conceptual vocabulary** to design, evaluate, and communicate EKA-based systems.

Let us begin.

0.1 The Problem That EKA Solves

Modern enterprises invest heavily in:

- Semantic modeling
  - OWL - Web Ontology Language,
  - RDF - Resource Description Framework,
  - SHACL - Shapes Constraint Language, and more

- Knowledge graphs (Neo4j as a property graph; GraphDB, Stardog as native RDF graph databases)
- Reasoning engines (Pellet, HermiT, RDFox)

Yet a common complaint remains:

> "We have ontologies and graphs, but our knowledge does not *act*."

Models are validated.

Graphs are queried.

But **knowledge is not executed** as part of automated, trustworthy, and adaptive business processes.

This gap is not accidental. Most semantic projects stop at one of three stages below:

Stopping Point	What is Missing?
Ontology only	Execution engine, dynamic query, rules as actions
Graph only	Formal semantics, logical consistency, inheritance
Rules only	Domain structure, classification, semantic governance

The gap has a name in current industry practice. *Meaning before Automation* -- a slogan associated with Frédéric Verhelst's *The Ontology Imperative* -- holds that agreed meaning is the precondition of automation, not an afterthought to it. EKA is the engineering answer to that slogan: it keeps the agreed meaning in the ontology and the knowledge graph layer (K), enforces it through reasoning (R), and makes it checkable in the governance layer (Γ) before anything in the execution layer (Φ) is allowed to act.

EKA (Executable Knowledge Architecture) is a framework designed to close the gap.

EKA does not replace ontologies, knowledge graphs, or reasoners.

Instead,

> it **orchestrates them** into a single, layered architecture where knowledge is not only represented but also executed.

0.2 EKA -- A Working Definition

0.2.1 Short Definition

> **EKA (Executable Knowledge Architecture)** is an architectural framework in which formal knowledge (ontologies, rules, constraints) is transformed into machine-executable intelligence through a structured pipeline:
>
> **Diagrams** $\rightarrow$ **Meta-models** $\rightarrow$ **Ontologies** $\rightarrow$ **Knowledge Graphs** $\rightarrow$ **Executable Intelligence**.

0.2.2 Formal Definition

Let us now provide a more precise, **formal definition** that can be used in research, tooling, and enterprise governance.

An **EKA system** is a tuple:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Where:

- K = Knowledge Graph layer - a set of entities (nodes) and semantic relationships (edges) conforming to an ontology.
- R = Reasoning & Rules layer - a set of inference rules (OWL, SWRL, or custom) that derive new facts from K.
- Θ = Trigger layer - a set of conditions (semantic events, queries, or time-based triggers) that initiate execution.
- Φ = Execution layer - a set of actions (API calls, process invocations, alerts, graph updates) that change the real world or the knowledge graph.

- Γ = Governance layer - constraints, validation rules (SHACL), and access policies that ensure semantic integrity.

An EKA system is **defined as executable if and only if (iff)** for every trigger $\theta \in \Theta$ that evaluates to `true`, at least one action $\phi \in \Phi$ is **automatically invoked** with guaranteed traceability.

> In plain language: EKA is not a static ontology. It is a **semantic automation architecture**.

> **Note**
>
> Note on Notation: Understanding $\in$
>
> Throughout this book, you will occasionally see the symbol $\in$ (pronounced "is an element of" or "belongs to"). Borrowed from set theory, it is simply a shorthand way to show that a specific item is part of a larger collection or set. For example, writing $\theta \in \Theta$ simply means that a given trigger (θ) is a member of the broader set of all possible triggers (Θ). Think of it like saying a specific menu item belongs to the restaurant's master menu—no advanced mathematics required!

0.3 The Five Layers of EKA (Implementation Roadmap)

The EKA roadmap appears throughout this book's diagrams. Let us now explain **each layer** in operational terms.

Layers	Artifact	Tool example	Question it answers
1. Diagramming	Informal conceptual sketches	Draw.io, Visio®, whiteboard	*What are the key concerns?*
2. Meta-modeling	Formal structural rules among concepts	ArchiMate®, UML (Unified Modeling Language) class diagrams, custom DSL (Domain-Specific Languages)	*How are concepts legally related?*
3. Ontology	OWL ontology with classes, properties, restrictions, disjointness	Protégé, Fluent Editor	*What is the formal meaning?*
4. Knowledge Graph (Choice 1: Native RDF)	Populated graph with individuals and inferred relationships	AllegroGraph, GraphDB, Stardog, RDFox, Amazon Neptune (RDF mode)	*What specific facts are true in a semantic, reasoner-compatible store?*
4. Knowledge Graph (Choice 2: Property Graph)	Transform graph for fast traversal	Neo4j, TigerGraph, JanusGraph, Amazon Neptune (property graph mode)	*How can we traverse relationships efficiently for specific queries?*
5. Executable Intelligence	Event-driven actions, queries, and decisions	*What should the system do now?*	

> **Important**
>
> **Critical Architectural Distinction**
>
> The "Knowledge Graph" layer in EKA can be implemented using two fundamentally different technologies:
>
> - **Native RDF Graph Databases** (AllegroGraph, Stardog, GraphDB, RDFox) store data as RDF triples, use global IRIs as identifiers, support SPARQL queries, and be able to perform OWL reasoning natively. This is the **recommended choice** for preserving and executing the full logical semantics of your OWL ontology.
> - **Property Graphs** (Neo4j, TigerGraph, JanusGraph) use internal, non-semantic element IDs and are optimized for fast graph traversal algorithms. An OWL ontology can be *transformed* into a property graph, but this is a **lossy transformation** that sacrifices OWL reasoning capabilities and semantic richness.

According to the formal EKA definition, the K **layer** can be implemented by either type, but the choice has significant architectural implications. For building a semantic layer over enterprise data that leverages reasoning and Linked Data, a Native RDF Graph Database is the correct architecture choice.

Furthermore, many projects claim to have an "EKA" because they have a knowledge graph. However, **without a trigger Θ and execution Φ, you do not have EKA** - you only have a semantic repository.

0.4 Why "Executable" Is Not the Same as "Reasoning"

A common misconception is that **OWL reasoning alone makes knowledge executable**.

Let us clarify.

Capability	OWL Reasoner	EKA Execution layer
Infers new class membership	✓ Yes	✓ Can use reasoner
Detects inconsistency	✓ Yes	✓ Yes
Triggers an external API	× No	✓ Yes
Sends an alert to a business system	× No	✓ Yes
Modifies a graph or database	Varies by system (see note below*)	✓ Yes
Operates over time (event-based)	× No	✓ Yes

Note

***Important Note on Inference Materialization**

In Protégé's default desktop workflow, reasoner results are normally shown as **inferred axioms** (displayed in the interface) rather than automatically saved as **asserted axioms** into the ontology file. This is a tool-specific behavior, not a universal principle of OWL reasoning or graph databases.

In production-grade semantic systems -- including triplestores, rule-enabled graph databases, and enterprise knowledge graph platforms -- inferred triples may be:

- **Materialized and persisted** (written back to storage),
- **Maintained as inferred closures** (updated incrementally), or
- **Exposed virtually** (computed on-the-fly at query time),

depending on the system architecture and configuration.

*The author thanks **Michael DeBellis** for his expert technical review and for clarifying this critical engineering distinction -- particularly the difference between Protégé's UI behavior and broader enterprise-grade inference materialization practices. (Ref: GitHub Issue #9 (https://github.com/yasenstar/protege_pizza/issues/9))*

OWL reasoning answers: *What else is true?* (Note: as mentioned just now, the reasoner does not directly write inferred triples back into the original OWL file or persistent triple store, unless you run Drools and write to OWL manually in Protégé.)

EKA execution answers: *What should be done now, automatically, based on what is true?*

In an EKA system, the reasoner is a **component** - not the whole engine.

The execution layer Φ is what makes knowledge **operational**.

0.5 EKA in Relation to Existing Standards and Architectures

To avoid confusion, it is helpful to position EKA relative to well-known frameworks.

Framework	Primary focus	Relationship to EKA
TOGAF - The OpenGroup Architecture Framework (ADM - Architecture Development Method)	Enterprise architecture process	EKA can be used as a **semantic automation pillar** within all TOGAF's ADM layers.
Semantic Web Stack (OWL, RDF, SPARQL - "SPARQL Protocol and RDF Query Language")	Knowledge representation and query	EKA **extends** the stack with an explicit execution layer.
Knowledge Graph (Native RDF: GraphDB, AllegroGraph, Stardog; Property Graph: Neo4j)	Graph storage and traversal	The K layer of EKA. Native RDF graphs are the recommended choice for preserving OWL semantics. Property graphs may be used for specialized, performance-intensive queries after transformation.
Rule engines (Drools, DMN - Decision Model and Notation)	Business rule execution	EKA generalizes rules into **semantic triggers + actions**
Data Fabric / Data Mesh	Data integration and decentralization	EKA adds **executable semantics** on top of integrated data.

EKA is **not** a replacement for any of these frameworks.

It is an **architectural pattern** that assembles them into a coherent, executable knowledge pipeline.

0.6 EKA Maturity Levels

Not every ontology project needs full EKA.

We define four maturity levels to help you decide.

Level	Name	Characteristics	When to use
L0	Semantic modeling	Protégé + OWL only, no graph, no execution	Learning, academic exploration
L1	Knowledge graph	Ontology + populated graph, queryable (SPARQL/Cypher)	Enterprise knowledge repository
L2	Reactive knowledge	L1 + trigger (Θ) and notifications	Monitoring, compliance, alerting
L3	Executable intelligence (full EKA)	L2 + autonomous actions (Φ) + Governance (Γ)	Autonomous AI, closed-loop architecture, digital twins

This book:

- Chapters 1-7 primarily cover **L0 and L1** (ontology + reasoning)
- Chapters 8-11 introduce **graph readiness** (L1-L2)
- Later chapters on SWRL, SPARQL, and external integrations build toward **L2 and L3**
- Especially, chapters 17-24 bring the Semantic Knowledge Development Lifecycle (SKDL) in the systematically way as the implementation steps for reaching EKA

You are not expected to build a full L3 system after reading the `Pizza.owl` tutorial.

But you **will** understand how each layer contributes to executability.

0.7 EKA and the `Pizza.owl` Tutorial

You may now ask: *Why does the `Pizza.owl` tutorial appear in an EKA book?*, or *"Why we link to EKA in this `Pizza.owl` video reference book?"*

The answer to both of them is strategic.

The `Pizza.owl` tutorial teaches:

- Classes, properties, restrictions (Ontology layer)
- Reasoning (preparation for R)
- Disjointness and consistency (part of Γ)

But the original tutorial **does not** cover:

- Exporting to a graph database (K)
- Defining triggers (Θ)
- Automating actions (Φ)

Therefore, this book uses `Pizza.owl` as the **common, repeatable, beginning-friendly domain** to teach **EKA concepts layer by layer**.

By the end of this book, you will have built:

1. A correct `Pizza.owl` ontology (L0)
2. A populated knowledge graph from it (L1)
3. Example triggers and actions (simulated or via API) (L2-L3)

What you learn from `Pizza` ontology scales to:

- Enterprise risk ontologies
- Supply chain knowledge graphs
- Healthcare eligibility rules
- Digital twin automation

The domain changes. The EKA architecture does not.

0.8 How to Use the EKA Definition in Your Work

As a reader of this book, you are encouraged to apply the EKA definition to your own projects.

Here is a simple **EKA readiness checklist** for any semantic project:

- **Ontology (K schema)**: Do we have an OWL ontology with classes, properties, and constraints?
- **Reasoning (R)**: Can a reasoner infer new relationships and detect inconsistencies?
- **Graph population (K instance)**: Is the knowledge stored in a queryable graph? **（Critical: Is it a Native RDF Graph that preserves OWL semantics, or a Property Graph that requires transformation?)**
- **Triggers (Θ)**: Are these conditions (semantic, temporal, or event-based) that start automation?
- **Actions (Φ)**: Does the system automatically invoke APIs, workflows, or alerts based on triggers?
- **Governance (Γ)**: Are validation, access control, and auditability enforced?

If you answer "no" to **Triggers** or **Actions**, your system is a **knowledge graph** -- valuable, but not yet EKA.

The goal is not always full EKA.

The goal is to **choose the right level** for your problem.

0.9 Key Concepts

Term	Definition
Property Graph	A graph database that uses a property graph data model with nodes, edges, and key-value properties. Uses internal, non-semantic element IDs. Examples include Neo4j, TigerGraph, JanusGraph, and Amazon Neptune (property graph mode). Property graphs are optimized for fast graph traversal algorithms but do not natively support OWL reasoning.
Native RDF Graph Database (RDF Triplestore)	A graph database that stores data natively as RDF triples. Uses global IRIs as identifiers, supports SPARQL queries, and can perform OWL reasoning directly. Examples include AllegroGraph, GraphDB, RDFox, Stardog, and Amazon Neptune (RDF mode). This is the recommended choice for the EKA K layer when preserving OWL semantics is required.
Knowledge Graph (in EKA context)	The populated (K) layer of an EKA system, representing a set of entities (nodes) and semantic relationships (edges) conforming to an ontology. Can be implemented using either a Native RDF Graph Database (preserving OWL semantics) or a Property Graph (with lossy transformation and significant trade-off).
Neo4j (clarification)	A popular property graph database. While it can be used as part of an EKA implementation, it requires a lossy transformation from OWL/RDF and does not natively support OWL reasoning. Distinguished from Native RDF Graph Databases like GraphDB, Stardog, and AllegroGraph.

0.10 Chapter (00) Summary

In this chapter (00), we formally defined the **Executable Knowledge Architecture (EKA)** that appears throughout this book.

You learned:

- The **problem** EKA solves (knowledge is modeled but not executed)
- A **short and formal definition** of EKA as a tuple (K, R, Θ, Φ, Γ)
- The **five-layer implementation roadmap** from diagrams to executable intelligence
- The **difference** between OWL reasoning and EKA execution
- How EKA relates to TOGAF, Semantic Web Stack, knowledge graphs, and rule engines.
- **Four maturity levels** (L0-L3) to right-size EKA adoption
- Why the `Pizza.owl` tutorial is the **vehicle** - not the destination - for learning EKA

From this point forward, whenever you see the **EKA Connection** section at the end of a chapter, you will recognize it as a specific layer, component, or maturity level being strengthened.

The next chapter (01) resumes the ontology journey -- but now you see the full architectural horizon.

0.11 Reference

- EKA Official Website: https://xiaoqi.com
- EKA Community Site in Github: https://github.com/EKA-Community/eka
- Michael Debellis, *A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins*
- W3C OWL 2 Specification: https://www.w3.org/TR/owl2-syntax/
- Neo4j Cypher Documentation (for EKA knowledge graph layer): https://neo4j.com/docs/cypher-manual/

Last updated at 2026-11-12